SpringCloud

- 服务治理
  - Eureka
    - 普通使用
      - 导包
      - 启动代码添加注解@EnableEurekaServer和@EnableDiscoveryClient（新版本不需要）
      - 写代码，用RestTemplate完成远程调用
        - 需要提前写一个@Configuration类，使用时注入RestTemplate的@Bean
        - 注入一个DiscoveryClient对象来获取IP和端口等信息
      - 配置文件(基础配置)
        - eureka.client.register-with-eureka： 表示是否将自己注册到Eureka Server，默认为true
        - eureka.client.fetch-registry： 表示是否从Eureka Server获取注册信息，默认为true
        - eureka.client.serviceUrl.defaultZone： 设置与Eureka Server交互的地址，查询服务和注册服务都需要依赖这个地址。默认是http://localhost:8761/eureka； 多个地址可使用 , 分隔
    - 集群
      - server.port=端口
      - eureka.instance.hostname=主机名
      - eureka.client.serviceUrl.defaultZone=http://主机名:端口/eureka/
      - hosts文件中加入127.0.0.1 主机名，防止使用外部DNS解析主机名
      - 打包启动
        - mvn clean package
        - java -jar spring-cloud-eureka-0.0.1-SNAPSHOT.jar --spring.profiles.active=主机名
  - consul
    - 同Eureka，配置稍有不同，同样使用RestTemplate调用服务
  - Nacos
    - 同Eureka，配置稍有不同，同样使用RestTemplate调用服务
- 客户端负载均衡
  - Ribbon
    - 在RestTemplate的@Configuration类中增加一个@LoadBalance注解
    - 写配置文件
    - 在启动类中加入@RibbonClient(name="", configuration=负载均衡配置类Rule.class)注解
      - 或者在配置文件里加入负载均衡规则设置
- Feign(声明式服务调用)
  - 服务消费者定义接口
  - 接口上添加注解@FeignClient(value="服务提供者应用名")
  - 编写调用接口，接口声明规则和提供方接口保持一致
  - 注入该接口对象，调用接口方法完成远程调用
  - 配置文件还能配置超时和日志等设置
- Hystrix熔断器
  - 隔离
  - 降级
    - 服务调用异常或超时则返回默认信息
    - 服务提供方降级
      - 导包
      - 在服务提供类的启动类中添加@EnableCircuitBreaker注解
      - Controller层中定义降级方法
        - 方法的返回值需要和原方法一样
        - 方法的参数需要和原方法一样
      - 给原方法增加一个@HystrixCommand(fallbackMethod="降级方法名")注解来关联降级方法
        - 还可以在注解中增加一个属性commandProperties={@HystrixProperty(name="设置属性名",value="值")}来进行配置
        - 例如execution.isolation.thread.timeoutInMilliseconds为超时毫秒设置
    - 服务消费方降级
      - Feign组件已经集成了Hystrix组件
      - 配置文件中开启对Hystrix的支持
      - 编写Feign接口的实现类用于降级处理
      - 使用@Component注解把此类加入Spring的IOC容器
      - 在Feign接口的@FeignClient注解中增加fallback属性=实现类.class
  - 熔断
    - 失败次数达到阈值，打开断路器，拒绝所有请求，默认开启5秒后进入半开状态，放入少量请求，如果调用成功次数到达阈值，关闭断路器
    - 配置可以在Controller层的@HystrixCommand注解中添加
      - circuitBreaker.sleepWindowInMilliseconds 监控时间，默认5000毫秒
      - circuitBreaker.requestVolumeThreshold 失败次数，默认20次
      - circuitBreaker.errorThresholdPercentage 失败率，默认50%
  - 限流
- Gateway网关
  - 静态路由（不建议）
  - 动态路由
    - 导包
    - 启动类
    - 配置
      - 增加eureka的客户端配置defaultZone
      - uri改成uri: lb://GATEWAY-CONSUMER这样类似的就行
  - 微服务名称配置
    - discovery.locator.enabled(lower-case-service-id)都设置为true即可不区分大小写支持微服务名称当路径访问
  - 过滤器
    - pre过滤器
      - 转发前执行
    - post过滤器
      - 响应前执行
    - GlobalFilter 全局过滤器
    - GatewayFilter 局部过滤器
    - 配置文件里配置就行，局部和全局就是创建实现类实现相应接口后编写逻辑
- config分布式配置中心
  - 使用流程
    - 配置Server
      - 导包
      - 编写启动类添加@EnableConfigServer注解
      - 编写配置文件，将外部的配置文件git库加入进来，如为私有库，则需要配置用户名密码
    - 配置Client
      - 步骤同上，差异在于配置文件，配置文件需要命名为bootstrap.yml，利用了其先加载的机制，再配置Server的uri、外部配置文件的name、profile、label等值，引入完成后用@Value("${}")注解即可注入相应名称的值
  - 客户端刷新
    - 导包actuator
    - 在获取配置信息类上添加@RefreshScope注解
    - 添加配置management.endpoints.web.exposure.include:refresh
    - 直接发个POST请求即可刷新 curl -X POST domain/actuator/refresh
      - 可以整合进Eureka中，配置文件将自己注册到Eureka中，Eureka Server中配置Discovery的service-id项即可
- Bus消息总线
  - 使用流程
    - 分别在Server和Client中导包bus-amqp
    - 分别在Server和Client中配置RabbitMQ
    - 在Server中设置暴露监控断点bug-refresh
      - 需要导包actuator
    - 发送POST请求来控制刷新即可
- Stream消息驱动
  - 用来统一操控消息中间件，当中间件替换时不需要大量改动
  - 消息生产者
    - 导包starter-stream-rabbit
    - 编写配置，定义binder和bindings
    - 启动类增加@EnableBinding(Source.class)注解
    - 注入 MessageChannel对象output(Bindings设置里的配置的名称)
    - 使用output.send(MessageBuilder.withPayload(message).build())方法发送消息
  - 消息消费者
    - 配置同上，指定binder和destination
    - 添加@StreamListener(Sink.INPUT)注解并使用message.getPayload()方法接收消息
- Sleuth+Zipkin链路追踪
  - 下载二进制程序直接运行即可 java -jar zipkin.jar
  - 默认访问9411端口
  - 在服务提供方和消费方引入相应的依赖然后编写配置文件即可